

Backend Recommendations — AutoSuivi.tn

Ce document liste les ajustements nécessaires ou recommandés sur le backend (`http://102.204.205.49/api`) pour garantir la compatibilité avec la nouvelle architecture Clean du frontend.

1. Format de réponse standardisé

Le frontend s'attend à un format de réponse cohérent. **Adopter un wrapper standard** pour toutes les réponses :

```
// Succès - objet unique
{
  "success": true,
  "vehicle": { ... }
}

// Succès - liste
{
  "success": true,
  "vehicles": [ ... ]
}

// Erreur
{
  "success": false,
  "message": "Description de l'erreur"
}
```

Note : L'ApiClient frontend est résilient et peut gérer plusieurs formats (`data.vehicle` , `data.data` , ou `data` directement), mais un format standard est préférable.

2. Endpoints requis

✓ Authentification (/api/auth/)

Méthode	Endpoint	Body	Réponse attendue
POST	/auth/login	{ email, password }	{ token, user: { id, email, name, created_at } }
POST	/auth/register	{ name, email, password }	{ token, user: { id, email, name, created_at } }
GET	/auth/profile	— (JWT header)	{ user: { id, email, name, created_at } }

✓ Véhicules (/api/vehicles/)

Méthode	Endpoint	Body	Réponse attendue
GET	/vehicles	—	{ vehicles: [...] }
GET	/vehicles/:id	—	{ vehicle: {...} }
POST	/vehicles	{ brand, model, year, plate, initial_mileage, current_mileage }	{ vehicle: {...} }
PUT	/vehicles/:id	champs partiels	{ vehicle: {...} }
DELETE	/vehicles/:id	—	{ message: "..." }

✓ Kilométrage (/api/vehicles/:vehicleId/mileage/)

Méthode	Endpoint	Body	Réponse attendue
POST	/vehicles/:vehicleId/mileage	{ value, date }	{ reading: {...} }
GET	/vehicles/:vehicleId/mileage	—	{ readings: [...] }

❑ Endpoints futurs (non bloquants)

Méthode	Endpoint	Description
GET	<code>/vehicles/:vehicleId/mileage/stats</code>	Stats calculées côté serveur
CRUD	<code>/vehicles/:vehicleId/main-tenance</code>	Enregistrements de maintenance
CRUD	<code>/vehicles/:vehicleId/dead-lines</code>	Échéances (assurance, vignette, visite technique)

3. Mapping des champs (snake_case)

Le backend doit utiliser **snake_case** pour les noms de champs dans les JSON. Le frontend convertit automatiquement :

Backend (snake_case)	Frontend (camelCase)
<code>user_id</code>	<code>userId</code>
<code>initial_mileage</code>	<code>initialMileage</code>
<code>current_mileage</code>	<code>currentMileage</code>
<code>is_active</code>	<code>isActive</code>
<code>created_at</code>	<code>createdAt</code>
<code>updated_at</code>	<code>updatedAt</code>
<code>vehicle_id</code>	<code>vehicleId</code>

4. Middleware JWT

Vérifier que le middleware d'authentification :

- Lit le token depuis le header `Authorization: Bearer <token>`
- Renvoie un statut **401** avec un body `{ message: "..."}` en cas de token invalide/expiré
- Injecte `req.user.id` pour filtrer les véhicules par utilisateur

5. Validation des données

Création de véhicule (POST /vehicles)

```
// Champs requis
brand: string (non vide)
model: string (non vide)
year: number (1900-2030)
plate: string (non vide, format tunisien de préférence)
initial_mileage: number (>= 0)
current_mileage: number (>= initial_mileage)
```

Ajout kilométrage (POST /vehicles/:vehicleId/mileage)

```
value: number (> 0, > dernière lecture)
date: string (ISO 8601, ex: "2024-03-28T14:30:00Z")
```

Le backend devrait :

- Calculer automatiquement le `delta` (différence avec la lecture précédente)
- Mettre à jour le `current_mileage` du véhicule après insertion

6. Gestion des erreurs

Codes HTTP recommandés :

- **200** : Succès
- **201** : Création réussie
- **400** : Données invalides (validation)
- **401** : Non authentifié / Token expiré
- **403** : Accès refusé (véhicule d'un autre utilisateur)
- **404** : Ressource non trouvée
- **500** : Erreur serveur

7. CORS

S'assurer que le serveur autorise les requêtes depuis :

- `http://localhost:8081` (Expo dev)
- `http://localhost:19006` (Expo web)
- Ou utiliser `Access-Control-Allow-Origin: *` en développement

8. Sécurité — Vérification d'appartenance

Pour chaque opération sur un véhicule, vérifier que `vehicle.user_id === req.user.id` :

```
// Middleware suggestion
const checkVehicleOwnership = async (req, res, next) => {
  const vehicle = await Vehicle.findByPk(req.params.id);
  if (!vehicle) return res.status(404).json({ message: 'Véhicule non trouvé' });
  if (vehicle.user_id !== req.user.id) return res.status(403).json({ message: 'Accès
refusé' });
  req.vehicle = vehicle;
  next();
};
```